

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 2: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 20 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false, multiple choice, and short answer). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** State the formal (c, n_0) definitions of $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$, and $f(n) = \Theta(g(n))$. Explain in one sentence each what they mean informally (upper bound, lower bound, tight bound).
- A2.** Prove, directly from the definition, that $f(n) = 5n^2 + 3n + 2$ satisfies $f(n) = O(n^2)$. Exhibit explicit constants c and n_0 .
- A3.** Prove, directly from the definition, that $f(n) = 5n^2 + 3n + 2$ satisfies $f(n) = \Omega(n^2)$. Exhibit explicit constants c and n_0 . Conclude that $f(n) = \Theta(n^2)$.
- A4.** Prove that Big-O is *reflexive* ($f(n) = O(f(n))$) and *transitive* (if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$).
- A5.** Prove that $f(n) = \Omega(g(n))$ if and only if $g(n) = O(f(n))$.
- A6.** True or false, with justification: if $f(n) = O(g(n))$, then $2^{f(n)} = O(2^{g(n)})$. (Hint: consider $f(n) = 2n$ and $g(n) = n$.)
- A7.** Order the following eight functions from slowest- to fastest-growing, and justify the relative order of any two consecutive functions in your list:
$$n^3, \quad \log_2 n, \quad n!, \quad n\sqrt{n}, \quad 2^n, \quad n^2 \log n, \quad n, \quad 100.$$
- A8.** Is $n^2 = O(n^3)$? Is $n^3 = O(n^2)$? Is $n^2 = \Theta(n^3)$? For each, answer yes/no and justify briefly using the definitions.
- A9.** For a polynomial $p(n) = a_d n^d + \dots + a_1 n + a_0$ with $a_d > 0$, prove that $p(n) = O(n^d)$. (You do not need to prove the matching $\Omega(n^d)$ bound, though you may for extra rigor.)
- A10.** Consider the code fragment

```

total = 0
for i in range(n):
    for j in range(i):
        total += 1

```

Derive an exact closed-form expression for the final value of `total` as a function of n , and state its tightest $\Theta(\cdot)$ bound.

- A11.** Consider the recurrence $T(n) = T(n-1) + 1$ for $n \geq 1$, with $T(0) = 1$. Solve it in closed form (by unrolling) and state its $\Theta(\cdot)$ class. Which algorithmic pattern does this recurrence correspond to?
- A12.** Consider the recurrence $T(n) = T(\lfloor n/2 \rfloor) + 1$ for $n \geq 1$, with $T(0) = 1$. Solve it in closed form (assuming $n = 2^k$ for simplicity) and state its $\Theta(\cdot)$ class. Which algorithmic pattern does this recurrence correspond to?
- A13.** Consider the recurrence $T(n) = 2T(n/2) + n$ for $n > 1$, with $T(1) = 1$. Using the recursion-tree method (sum the work done at each level of recursion), show that $T(n) = \Theta(n \log n)$.
- A14.** Define what it means for an algorithm to run in *polynomial time*. Explain why “polynomial time” is considered a reasonable (if imperfect) threshold for “efficient”, giving one argument based on *composability* of polynomial-time subroutines.
- A15.** Give a concrete numerical example (a table of running times for $n = 10, 30, 50$, say) that illustrates the practical gap between an $O(n^3)$ algorithm and an $O(2^n)$ algorithm. At what rough value of n does the $O(2^n)$ algorithm become impractical (say, $> 10^9$ operations)?
- A16.** State the loop invariant used to prove the correctness of iterative binary search (`BINARYSEARCHITERATIVE`). Explain the three parts of a loop-invariant proof (initialization, maintenance, termination) as applied to this invariant.
- A17.** Binary search shrinks the candidate range by (at least) half on every iteration. Use this fact to derive the $\Theta(\log n)$ bound on the number of iterations, being careful about the “at least half” vs. “exactly half” distinction (the range size is $\lceil m/2 \rceil$ or $\lfloor m/2 \rfloor$ depending on which half is kept, where m is the previous range size).
- A18.** Explain the “divide into pairs” trick for finding both the max and the min of n elements using roughly $\frac{3n}{2}$ comparisons instead of $2(n-1)$. Why are both approaches $\Theta(n)$, and in what sense is the paired approach still an improvement?
- A19.** Explain the amortized-analysis argument for why appending to a dynamic array that doubles its capacity on overflow costs $O(1)$ *amortized* per append, even though individual appends that trigger a resize cost $\Theta(n)$. Your answer should mention the total number of element copies over n appends.
- A20. (Tie-in to Week 1: Gale–Shapley complexity).** Week 1’s dictionary-based Gale–Shapley implementation used `.index()` to find a person’s rank in a preference list, where `.index()` on a Python list of length n is $O(n)$. The algorithm’s main loop runs $O(n^2)$ times (Week 1, Lemma on proposals), and each iteration may call `.index()` a constant number of times. What is the resulting worst-case running time of this implementation, as a function of n ? Now suppose (as in this week’s Problem 16) we precompute a rank table `women_rank[w][m]`

once, in $O(n^2)$ time, so that each rank lookup becomes $O(1)$. What is the running time of the algorithm using this precomputed table? Explain why precomputing the rank table is worth the $O(n^2)$ upfront cost.

Part B: Quiz

- B1. (True/False)** If $f(n) = \Theta(g(n))$, then $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- B2. (Short answer)** What is the tightest $\Theta(\cdot)$ bound for $f(n) = 7 \log n + n^{0.5}$?
- B3. (Multiple choice)** Which of the following is the *tightest* valid asymptotic bound for $f(n) = n^2 + 1000n$?
- (a) $O(n)$
 - (b) $O(n \log n)$
 - (c) $\Theta(n^2)$
 - (d) $\Theta(n^3)$
- B4. (True/False)** $2^{n+1} = \Theta(2^n)$.
- B5. (Short answer)** For the recurrence $T(n) = T(n/2) + 1$, $T(1) = 1$, what is $T(1024)$?
- B6. (Multiple choice)** A loop `for i in range(n): for j in range(i, n): ...` (with $O(1)$ work per inner iteration) runs the inner body a total of:
- (a) $\Theta(n)$ times
 - (b) $\Theta(n \log n)$ times
 - (c) $\Theta(n^2)$ times
 - (d) $\Theta(n^3)$ times
- B7. (True/False)** An algorithm with worst-case running time $O(n^{50})$ is, by the definition used in this course, a polynomial-time algorithm.
- B8. (Short answer)** What is $\lceil \log_2(16 + 1) \rceil$, the worst-case number of recursive calls for binary search on a sorted array of length $n = 16$?
- B9. (True/False)** For the dynamic-array doubling strategy, the total number of element copies after $n = 17$ appends (starting from capacity 1) is strictly less than $2n = 34$.
- B10. (Short answer)** In the efficient array/queue-based Gale–Shapley implementation (Problem 16), what is the time complexity of `build_women_rank`, as a function of n ?

Part C: Solutions

Part A

- A1.** $f(n) = O(g(n))$: there exist constants $c > 0, n_0 \geq 0$ such that $f(n) \leq c g(n)$ for all $n \geq n_0$ (f grows no faster than g , an upper bound). $f(n) = \Omega(g(n))$: there exist $c > 0, n_0 \geq 0$ such that $f(n) \geq c g(n)$ for all $n \geq n_0$ (f grows at least as fast as g , a lower bound). $f(n) = \Theta(g(n))$: both of the above hold (f and g grow at the same rate, a tight bound).
- A2.** Take $c = 6$ and $n_0 = 4$. For $n \geq 4$: $3n + 2 \leq 3n + 2n = 5n \leq n \cdot n = n^2$ (using $3n + 2 \leq 5n$ for $n \geq 1$, and $5n \leq n^2$ for $n \geq 5$; to be safe take $n_0 = 5$). Then $f(n) = 5n^2 + (3n + 2) \leq 5n^2 + n^2 = 6n^2 = c n^2$ for $n \geq n_0 = 5$. Hence $f(n) = O(n^2)$ with $c = 6, n_0 = 5$.
- A3.** Take $c = 5$ and $n_0 = 0$. For all $n \geq 0$, $f(n) = 5n^2 + 3n + 2 \geq 5n^2 = c \cdot n^2$ (since $3n + 2 \geq 0$). Hence $f(n) = \Omega(n^2)$ with $c = 5, n_0 = 0$. Combined with A2, $f(n) = O(n^2)$ and $f(n) = \Omega(n^2)$, so $f(n) = \Theta(n^2)$.
- A4. Reflexivity:** take $c = 1, n_0 = 0$; then $f(n) \leq 1 \cdot f(n)$ trivially for all $n \geq 0$, so $f(n) = O(f(n))$.
Transitivity: Suppose $f(n) = O(g(n))$, so there exist c_1, n_1 with $f(n) \leq c_1 g(n)$ for $n \geq n_1$. Suppose $g(n) = O(h(n))$, so there exist c_2, n_2 with $g(n) \leq c_2 h(n)$ for $n \geq n_2$. Let $n_0 = \max(n_1, n_2)$ and $c = c_1 c_2$. For $n \geq n_0$: $f(n) \leq c_1 g(n) \leq c_1 (c_2 h(n)) = c h(n)$. Hence $f(n) = O(h(n))$.
- A5.** ($\Rightarrow$) If $f(n) = \Omega(g(n))$, there exist $c > 0, n_0$ with $f(n) \geq c g(n)$ for $n \geq n_0$, i.e. $g(n) \leq \frac{1}{c} f(n)$ for $n \geq n_0$. Taking $c' = 1/c > 0$, this is exactly the statement $g(n) = O(f(n))$. ($\Leftarrow$) Symmetric: if $g(n) = O(f(n))$, there exist $c > 0, n_0$ with $g(n) \leq c f(n)$ for $n \geq n_0$, i.e. $f(n) \geq \frac{1}{c} g(n)$ for $n \geq n_0$, giving $f(n) = \Omega(g(n))$ with constant $1/c$.
- A6. False.** Take $f(n) = 2n$ and $g(n) = n$; clearly $f(n) = O(g(n))$ (with $c = 2$). But $2^{f(n)} = 2^{2n} = (2^n)^2 = (2^{g(n)})^2$. For $2^{2n} = O(2^n)$ we would need a constant c with $2^{2n} \leq c \cdot 2^n$ for all large n , i.e. $2^n \leq c$ – impossible since $2^n \rightarrow \infty$. So $2^{f(n)} \neq O(2^{g(n)})$ in general: Big-O is *not* preserved under exponentiation.
- A7.** From slowest- to fastest-growing:

$$100 \prec \log_2 n \prec n \prec n\sqrt{n} \prec n^2 \log n \prec n^3 \prec 2^n \prec n!.$$

Justifications: $100 \prec \log_2 n$ since $\log_2 n \rightarrow \infty$ while 100 is constant. $\log_2 n \prec n$ since $\log n = o(n)$. $n \prec n\sqrt{n} = n^{1.5}$ trivially (larger exponent). $n^{1.5} \prec n^2 \log n$ since $n^2 \log n / n^{1.5} = n^{0.5} \log n \rightarrow \infty$. $n^2 \log n \prec n^3$ since $n^3 / (n^2 \log n) = n / \log n \rightarrow \infty$. $n^3 \prec 2^n$ since every polynomial is $o(2^n)$. $2^n \prec n!$ since $n! / 2^n = \prod_{i=1}^n (i/2) \rightarrow \infty$ (eventually every factor exceeds 1).

- A8.** $n^2 = O(n^3)$: **yes**. Take $c = 1, n_0 = 1$; for $n \geq 1$, $n^2 \leq n^3$. $n^3 = O(n^2)$: **no**. If it held, there would be c, n_0 with $n^3 \leq c n^2$, i.e. $n \leq c$ for all $n \geq n_0$ – impossible since $n \rightarrow \infty$. $n^2 = \Theta(n^3)$: **no**, since Θ requires both O and Ω , and $n^2 = \Omega(n^3)$ would require $n^2 \geq c n^3$, i.e. $1/n \geq c$ for all large n – impossible for $c > 0$.
- A9.** For $n \geq 1$, each term satisfies $|a_i| n^i \leq |a_i| n^d$ (since $i \leq d$ and $n \geq 1$ implies $n^i \leq n^d$). Hence $p(n) \leq \sum_{i=0}^d |a_i| n^i \leq \left(\sum_{i=0}^d |a_i| \right) n^d$. Taking $c = \sum_{i=0}^d |a_i|$ and $n_0 = 1$ gives $p(n) \leq c n^d$ for all $n \geq 1$, so $p(n) = O(n^d)$.

- A10.** The inner loop runs i times for each value of $i = 0, 1, \dots, n-1$, so **total** $= \sum_{i=0}^{n-1} i = \frac{(n-1)n}{2} = \binom{n}{2}$. This is $\Theta(n^2)$.
- A11.** Unrolling: $T(n) = T(n-1) + 1 = T(n-2) + 2 = \dots = T(0) + n = 1 + n$. So $T(n) = n + 1 = \Theta(n)$. This is the pattern of **linear recursion** – e.g. a simple recursive countdown or a single recursive pass over a list/array (one recursive call per element).
- A12.** Assuming $n = 2^k$: $T(2^k) = T(2^{k-1}) + 1 = \dots = T(2^0) + k = T(1) + k = 1 + k = 1 + \log_2 n$. So $T(n) = \Theta(\log n)$. This is the pattern of **halving recursion** – e.g. binary search, or any algorithm that discards half its input at each step.
- A13.** Assume $n = 2^k$. At recursion depth i ($i = 0, \dots, k-1$), there are 2^i sub-problems, each of size $n/2^i$. The non-recursive (“+ n ” at the top level, or proportionally $+(n/2^i)$ per sub-problem at depth i) work at depth i totals $2^i \cdot \frac{n}{2^i} = n$. Summing over $k = \log_2 n$ levels (depth 0 through $k-1$), plus $\Theta(n)$ base cases at depth k (each of size 1, $T(1) = 1$, and there are n of them), the total is $n \cdot \log_2 n + \Theta(n) = \Theta(n \log n)$.
- A14.** An algorithm runs in **polynomial time** if its worst-case running time is $O(n^k)$ for some constant k independent of the input. **Composability argument:** if algorithm A runs in $O(n^a)$ and calls, as a subroutine, algorithm B running in $O(n^b)$, a polynomial (e.g. $O(n^c)$) number of times, the overall running time is $O(n^c \cdot n^b) + O(n^a) = O(n^{b+c}) + O(n^a)$, still polynomial. So polynomial-time algorithms can be freely composed without “escaping” the polynomial-time class – unlike, say, composing two exponential algorithms (which could give a doubly-exponential result).

	Running time	$n = 10$	$n = 30$	$n = 50$
A15.	n^3	1,000	27,000	125,000
	2^n	1,024	1,073,741,824	$\approx 1.13 \times 10^{15}$

At $n = 30$, $O(n^3)$ is already over 10^4 times smaller than $O(2^n)$. $O(2^n)$ exceeds 10^9 operations once $n \gtrsim 30$ (since $2^{30} \approx 1.07 \times 10^9$), while $O(n^3)$ does not reach 10^9 until $n \gtrsim 1000$. So the $O(2^n)$ algorithm becomes impractical around $n \approx 30$ –40, while the $O(n^3)$ algorithm remains usable for n in the thousands.

- A16.** The invariant is: “if x is present in A , its index lies in $[lo, hi]$.” **Initialization:** before the first iteration, $[lo, hi] = [0, n-1]$, which trivially contains every index, so the invariant holds. **Maintenance:** if $A[mid] \neq x$, sortedness of A guarantees that if x is present it must be on the side of mid consistent with the comparison $A[mid]$ vs. x ; updating lo or hi accordingly keeps x ’s index (if it exists) inside the new $[lo, hi]$. **Termination:** the loop ends either by finding x at mid (correct), or when $lo > hi$ – by the invariant, an empty range means x is not present, so returning -1 is correct.
- A17.** Let m_k be the range size before the k -th iteration, $m_0 = n$. Each iteration checks $A[mid]$ and keeps one of the two halves, each of size at most $\lceil (m_k - 1)/2 \rceil \leq \lceil m_k/2 \rceil$, so $m_{k+1} \leq \lceil m_k/2 \rceil$. Repeatedly halving (rounding up) from n reaches 0 after at most $\lceil \log_2(n+1) \rceil$ steps – more precisely, $m_k \leq \lceil n/2^k \rceil$, and the loop must terminate once $m_k = 0$, which happens for $k \geq \log_2(n+1)$. Hence the iteration count is $\Theta(\log n)$, with the precise worst case $\lceil \log_2(n+1) \rceil$.
- A18.** For each pair (a, b) of the n elements (assume n even for simplicity, $n/2$ pairs): 1 comparison decides which of a, b is the pair’s local max and which is the local min. Then 2 more comparisons update the running max (against the local max) and running min (against the local

min). Total: $3 \cdot (n/2) = \frac{3n}{2}$ comparisons, versus the naive $2(n-1) \approx 2n$ for two separate scans. Both are $\Theta(n)$ – the asymptotic class is unchanged – but $\frac{3n}{2} < 2n$ for all $n > 0$, so the paired approach uses fewer comparisons by a constant factor (a real, if modest, improvement within the same Θ class).

- A19.** Resizes (doublings) happen when the size, just before an append, equals the current capacity, i.e. at sizes $1, 2, 4, 8, \dots, 2^{k-1}$ where $2^k \geq n$ is the final capacity. The i -th resize (from capacity 2^{i-1} to 2^i) copies 2^{i-1} elements. Total copies $= 1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1 < 2n$ (since $2^{k-1} < n \leq 2^k$ implies $2^k < 2n$). So the total cost of copying across all n appends is $O(n)$, even though a single resize costs $\Theta(n)$. Dividing the total $O(n)$ cost over n appends gives $O(1)$ *amortized* cost per append – the expensive resizes are “paid for” by the many cheap appends between them.
- A20.** With `.index()` lookups (each $O(n)$), and $O(n^2)$ total loop iterations each doing $O(1)$ such lookups, the worst-case running time is $O(n^2) \times O(n) = O(n^3)$. With a precomputed rank table (built in $O(n^2)$ time – filling an $n \times n$ table, one entry per (woman, rank) pair), each lookup becomes $O(1)$, so the main loop costs $O(n^2) \times O(1) = O(n^2)$, plus the $O(n^2)$ precomputation, for a total of $O(n^2)$ – asymptotically better than $O(n^3)$ for large n . The upfront $O(n^2)$ cost is worth it because it is *dominated by* (same order as, in fact smaller in practice than) the main loop’s cost, and it eliminates a per-iteration $O(n)$ factor that would otherwise dominate everything.

Part B

- B1. True.** This is exactly the definition of Θ : $f(n) = \Theta(g(n))$ means $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- B2.** $\Theta(n^{0.5})$ (equivalently $\Theta(\sqrt{n})$). The $\sqrt{n}$ term dominates $\log n$ asymptotically (any positive power of n dominates $\log n$), so the sum is $\Theta(\sqrt{n})$.
- B3. (c)** $\Theta(n^2)$. The n^2 term dominates the lower-order $1000n$ term; 1000 is just a constant absorbed into the Θ .
- B4. True.** $2^{n+1} = 2 \cdot 2^n$, so taking $c_1 = c_2 = 2$ (and $n_0 = 0$) gives $2 \cdot 2^n \leq 2^{n+1} \leq 2 \cdot 2^n$ trivially (it’s exactly $2 \cdot 2^n$), so $2^{n+1} = \Theta(2^n)$.
- B5.** $T(1024) = T(2^{10}) = T(1) + 10 = 1 + 10 = 11$ (using $T(n) = 1 + \log_2 n$ from A11).
- B6. (c)** $\Theta(n^2)$. The inner loop runs $n - i$ times for each i , so the total is $\sum_{i=0}^{n-1} (n - i) = \sum_{j=1}^n j = \frac{n(n+1)}{2} = \Theta(n^2)$ – same order as a full nested loop, just with a smaller constant ($\approx n^2/2$ instead of n^2).
- B7. True.** 50 is a constant, so $O(n^{50})$ is of the form $O(n^k)$ for fixed k , which is exactly the definition of polynomial time (even though it is impractical for real use, as discussed in A15’s caveats).
- B8.** $\lceil \log_2 17 \rceil = 5$ (since $2^4 = 16 < 17 \leq 32 = 2^5$).
- B9. True.** From A19, the total copies after n appends is $2^k - 1$ where 2^k is the smallest power of two $\geq n$. For $n = 17$, $2^k = 32$, so total copies $= 31 < 34 = 2 \times 17$.

B10. $\Theta(n^2)$. `build_women_rank` fills an $n \times n$ table `women_rank[w][m]`, doing $O(1)$ work per (woman, man) pair via one pass over each woman's preference list of length n , for a total of $n \times n = \Theta(n^2)$.